School of Computer Sciences and IT
Institute of Management Sciences, Peshawar

Ride Sharing System

Submitted By: Muhammad Musa
Muhammad Zikria Shah

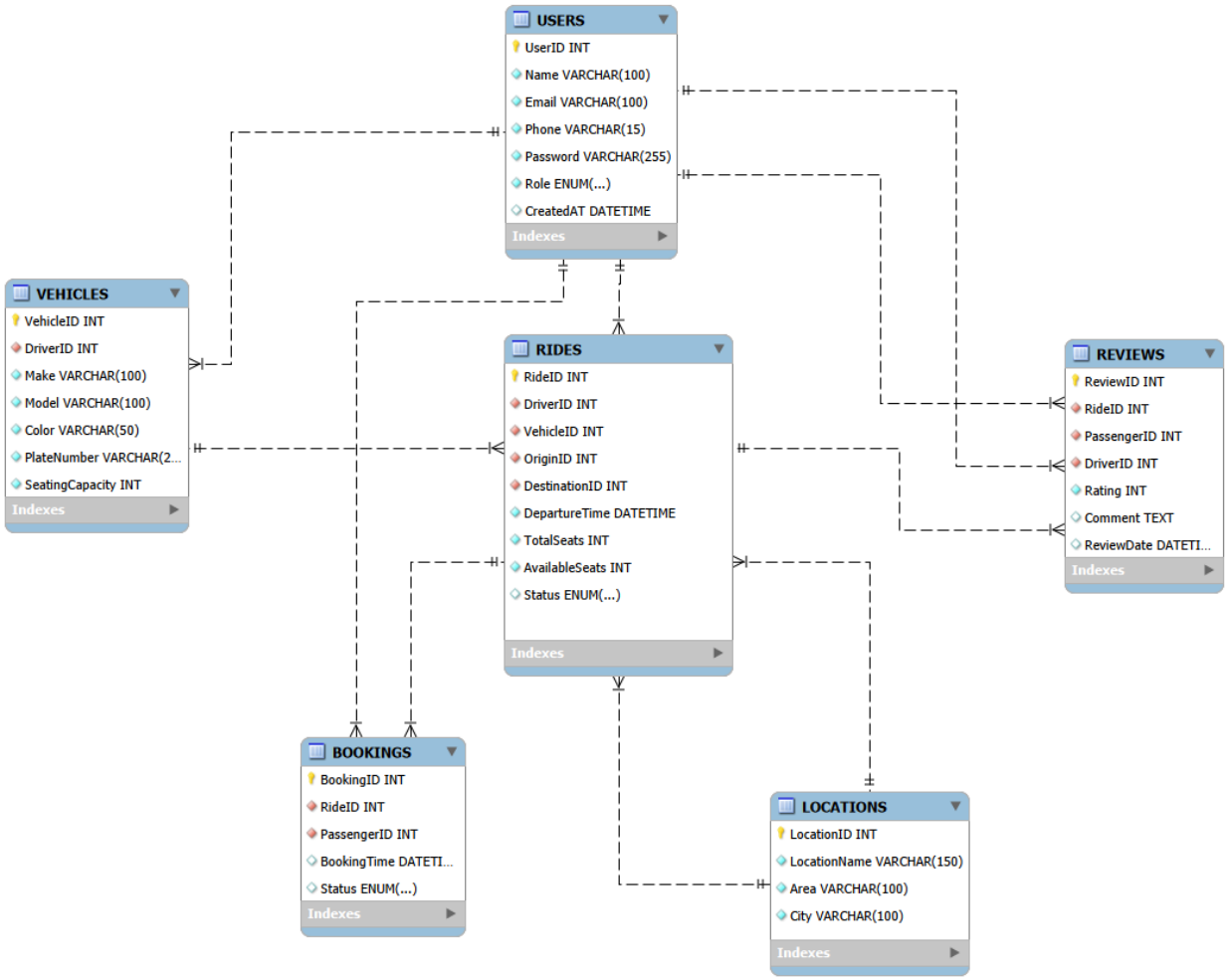
Submitted To: Sir Ali Hassan

Date: 2026-04-24

Project Milestones

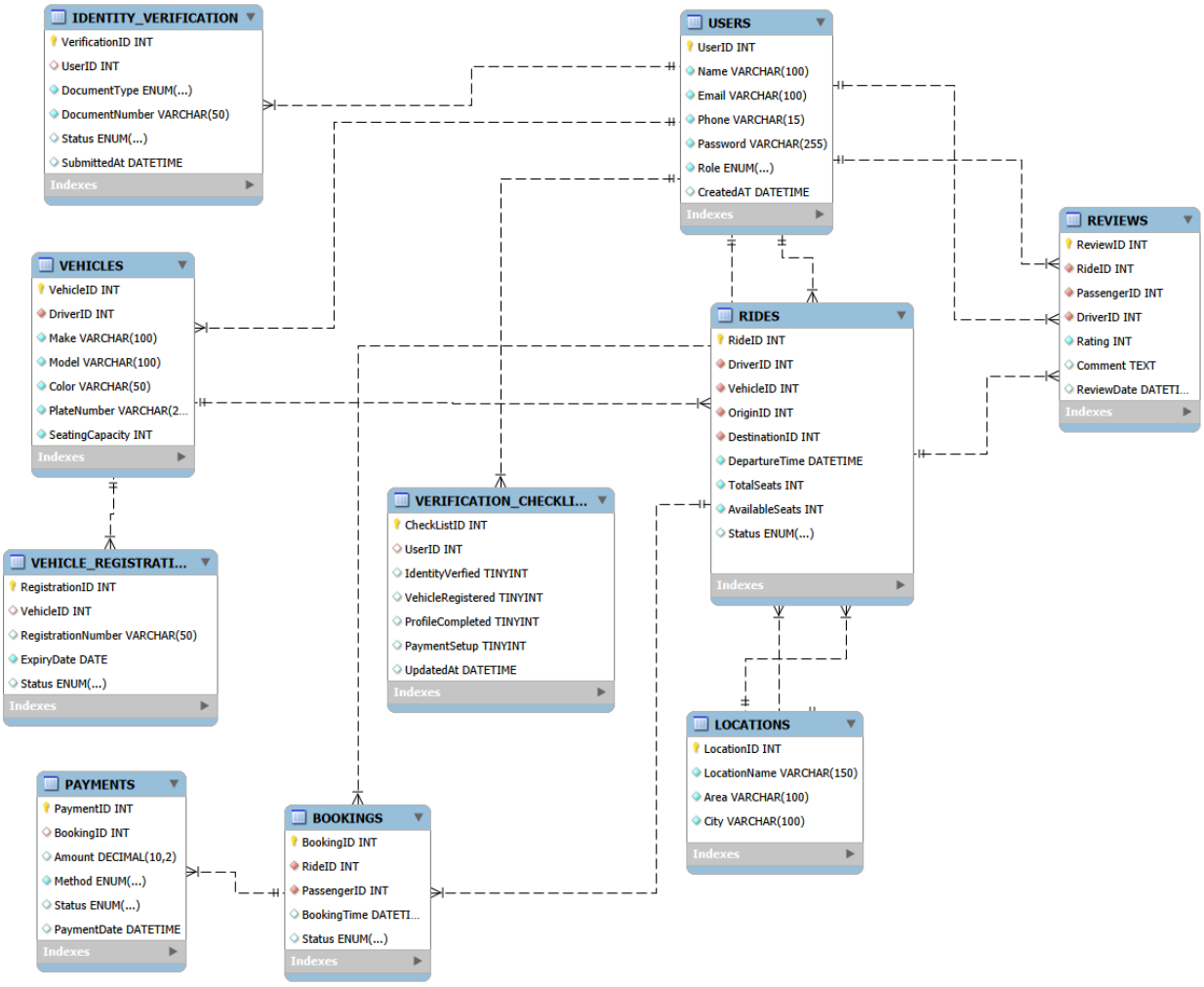
Milestone	Version	Date	Remarks
Milestone 1 – ERD & Relational Schema	V1.0	26/04/2026	
Milestone 1 – Schema Update	V1.1	11/05/2026	
Milestone 2 – Normalization	V2.0	11/05/2026	
Milestone 3 – Synthetic Data and Data Flow	V3.0	11/05/2026	

1. Entity-Relationship Diagram (ERD) (V1.0)



Entity-Relationship Diagram – Ride Sharing System

1. Entity-Relationship Diagram (ERD) (V1.1)



2. Relational Database Schema

This section presents the relational schema derived from the Entity-Relationship Diagram (ERD). Each entity is transformed into a table with clearly defined primary keys (PK) and foreign keys (FK) to maintain relationships and data integrity.

2.1 USERS

Description: Stores all system users, including both drivers and passengers.

```
USERS( UserID PK, Name, Email, Phone, Password, Role, CreatedAt )
```

	Field	Type	Null	Key	Default	Extra
▶	UserID	int	NO	PRI	NULL	auto_increment
	Name	varchar(100)	NO		NULL	
	Email	varchar(100)	NO	UNI	NULL	
	Phone	varchar(15)	NO		NULL	
	Password	varchar(255)	NO		NULL	
	Role	enum('Driver','Passenger')	NO		NULL	
	CreatedAt	datetime	NO		NULL	

USERS Table

2.2 VEHICLES

Description: Contains vehicle details associated with drivers.

```
VEHICLES( VehicleID PK, DriverID FK -> USERS(UserID), Make, Model, Color,
PlateNumber, SeatingCapacity )
```

	Field	Type	Null	Key	Default	Extra
▶	VehicleID	int	NO	PRI	NULL	auto_increment
	DriverID	int	NO	MUL	NULL	
	Make	varchar(100)	NO		NULL	
	Model	varchar(100)	NO		NULL	
	Color	varchar(50)	NO		NULL	
	PlateNumber	varchar(20)	NO		NULL	
	SeatingCapacity	int	NO		NULL	

VEHICLES Table

2.3 LOCATIONS

Description: Stores pickup and destination location details.

```
LOCATIONS( LocationID PK, LocationName, Area, City )
```

	Field	Type	Null	Key	Default	Extra
▶	LocationID	int	NO	PRI	NULL	auto_increment
	LocationName	varchar(150)	NO		NULL	
	Area	varchar(100)	NO		NULL	
	City	varchar(100)	NO		NULL	

LOCATIONS Table

2.4 RIDES

Description: Represents rides created by drivers, including route, timing, and seat availability.

```
RIDES( RideID PK, DriverID FK -> USERS(UserID), VehicleID FK ->
VEHICLES(VehicleID), OriginID FK -> LOCATIONS(LocationID), DestinationID FK ->
LOCATIONS(LocationID),DepartureTime, TotalSeats, AvailableSeats, Status )
```

Field	Type	Null	Key	Default	Extra
RideID	int	NO	PRI	NULL	auto_increment
DriverID	int	NO	MUL	NULL	
VehicleID	int	NO	MUL	NULL	
OriginID	int	NO	MUL	NULL	
DestinationID	int	NO	MUL	NULL	
RideTime	date	NO		NULL	
DepartureTime	time	NO		NULL	
TotalSeats	int	NO		NULL	
AvailableSeats	int	NO		NULL	
Status	enum('Scheduled','Ongoing','Completed')	NO		NULL	

RIDES Table

2.5 BOOKINGS

Description: Stores ride reservations made by passengers.

```
BOOKINGS( BookingID PK, RideID FK -> RIDES(RideID), PassengerID FK ->
USERS(UserID), BookingTime, Status )
```

Field	Type	Null	Key	Default	Extra
BookingID	int	NO	PRI	NULL	auto_increment
RideID	int	NO	MUL	NULL	
PassengerID	int	NO	MUL	NULL	
BookingTime	datetime	NO		NULL	
Status	enum('Pending','Confirmed','Cancelled')	NO		NULL	

BOOKINGS Table

2.6 REVIEWS

Description: Stores feedback and ratings given by passengers to drivers after rides.

```
REVIEWS( ReviewID PK, RideID FK -> RIDES(RideID), PassengerID FK ->
USERS(UserID), DriverID FK -> USERS(UserID), Rating, Comment, ReviewDate )
```

Field	Type	Null	Key	Default	Extra
ReviewID	int	NO	PRI	NULL	auto_increment
RideID	int	NO	MUL	NULL	
PassengerID	int	NO	MUL	NULL	
DriverID	int	NO	MUL	NULL	
Rating	int	NO		NULL	
Comment	text	YES		NULL	
ReviewDate	datetime	NO		NULL	

REVIEWS Table

2.7 PAYMENTS

Description: Stores payment transactions made by passengers for their ride bookings.

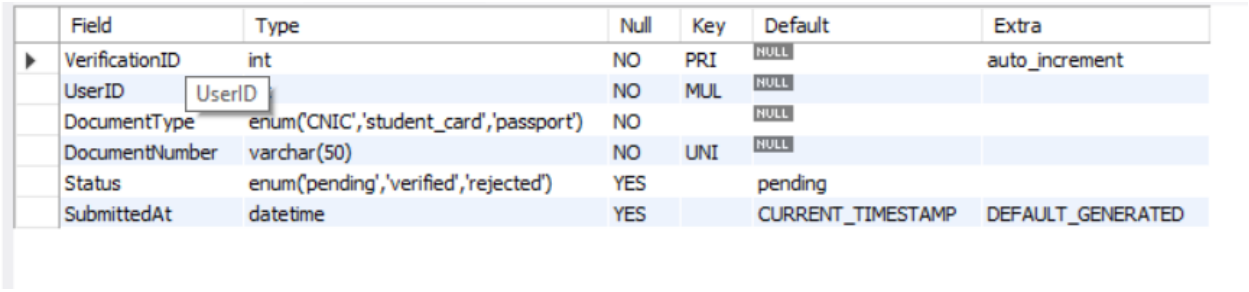
```
PAYMENTS( PaymentID PK, BookingID FK -> BOOKINGS(BookingID), Amount, Method,
                                Status, PaymentDate )
```

Field	Type	Null	Key	Default	Extra
PaymentID	int	NO	PRI	NULL	auto_increment
BookingID	int	NO	MUL	NULL	
Amount	decimal(10,2)	NO		NULL	
Method	enum('cash','online')	NO		NULL	
Status	enum('pending','completed','failed')	YES		pending	
PaymentDate	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

2.8 IDENTITY_VERIFICATION

Description: Stores identity document submissions by users for verification purposes.

```
IDENTITY_VERIFICATION( VerificationID PK, UserID FK -> USERS(UserID),
DocumentType, DocumentNumber, Status, SubmittedAt )
```

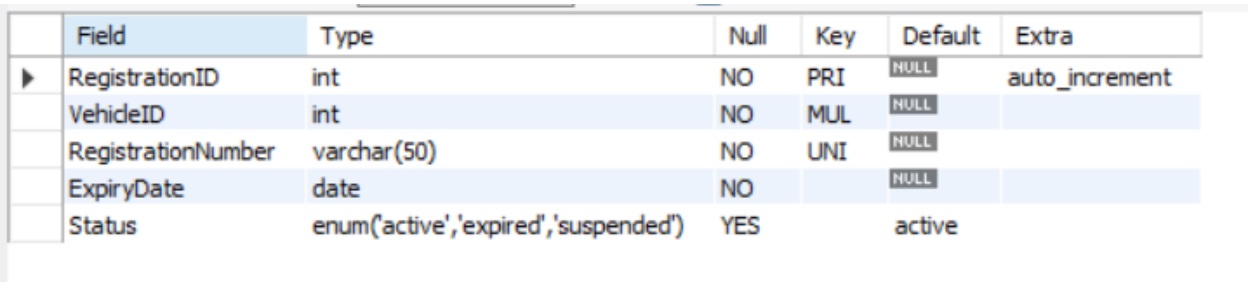


Field	Type	Null	Key	Default	Extra
VerificationID	int	NO	PRI	NULL	auto_increment
UserID	UserID	NO	MUL	NULL	
DocumentType	enum('CNIC','student_card','passport')	NO		NULL	
DocumentNumber	varchar(50)	NO	UNI	NULL	
Status	enum('pending','verified','rejected')	YES		pending	
SubmittedAt	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

2.9 IDENTITY_VERIFICATION

Description: Stores vehicle registration details linked to each registered vehicle.

```
VEHICLE_REGISTRATION( RegistrationID PK, VehicleID FK -> VEHICLES(VehicleID),
RegistrationNumber, ExpiryDate, Status )
```



Field	Type	Null	Key	Default	Extra
RegistrationID	int	NO	PRI	NULL	auto_increment
VehicleID	int	NO	MUL	NULL	
RegistrationNumber	varchar(50)	NO	UNI	NULL	
ExpiryDate	date	NO		NULL	
Status	enum('active','expired','suspended')	YES		active	

2.10 VERIFICATION_CHECKLIST

Description: Tracks the verification progress of each user across multiple checklist items.

VERIFICATION_CHECKLIST(CheckListID PK, UserID FK -> USERS(UserID), IdentityVerified, VehicleRegistered, ProfileCompleted, PaymentSetup, UpdatedAt)

Field	Type	Null	Key	Default	Extra
CheckListID	int	NO	PRI	NULL	auto_increment
UserID	int	NO	MUL	NULL	
IdentityVerified	tinyint	YES		0	
VehicleRegistered	tinyint	YES		0	
ProfileCompleted	tinyint	YES		0	
PaymentSetup	tinyint	YES		0	
UpdatedAt	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Normalization — Ride Sharing System

Database: mydb | Version: 1.1 | Date: 11-05-2026

This document presents the normalization analysis for all 10 tables in the Ride Sharing System database. Each table is evaluated against First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). Where a table already satisfies a normal form, a written justification is provided explaining why no change was needed.

---

1. USERS

1NF: The USERS table is in First Normal Form. All columns contain atomic values — each column stores a single value per row. There are no repeating groups or multi-valued attributes. UserID serves as the unique primary key.

2NF: The USERS table is in Second Normal Form. It has a single-column primary key (UserID), therefore partial dependency is not possible. All non-key attributes (Name, Email, Phone, Password, Role, CreatedAT) are fully dependent on UserID.

3NF: The USERS table is in Third Normal Form. No non-key column depends on another non-key column. Every attribute (Name, Email, Phone, Password, Role, CreatedAT) depends directly and only on UserID. No transitive dependencies exist.

Changes Made: None. The table already satisfies 1NF, 2NF, and 3NF.

---

2. VEHICLES

1NF: All columns are atomic with single values per row. VehicleID is the unique primary key. No repeating groups exist.

2NF: Single-column primary key (VehicleID), so partial dependency is not possible. All attributes fully depend on VehicleID.

3NF: No transitive dependencies. Make, Model, Color, PlateNumber, and SeatingCapacity all depend directly on VehicleID, not on each other.

Changes Made: None.

---

3. LOCATIONS

1NF: All columns (LocationName, Area, City) are atomic. LocationID is the unique primary key.

2NF: Single-column primary key (LocationID), partial dependency not applicable.

3NF: Area and City could be argued as dependent on each other in the real world, but in this system they are treated as independent attributes of a location, both depending directly on LocationID. No transitive dependency exists within the defined schema.

Changes Made: None.

---

4. RIDES

1NF: All columns are atomic. RideID is the unique primary key. No multi-valued attributes exist.

2NF: Single-column primary key (RideID), partial dependency not applicable. All attributes including DriverID, VehicleID, OriginID, DestinationID, DepartureTime, TotalSeats, AvailableSeats, and Status fully depend on RideID.

3NF: No non-key column depends on another non-key column. AvailableSeats could theoretically be derived from TotalSeats minus booked seats, but it is stored explicitly for performance purposes and depends on RideID directly. No transitive dependencies exist.

Changes Made: None.

---

5. BOOKINGS

1NF: All columns are atomic. BookingID is the unique primary key. No repeating groups exist.

2NF: Single-column primary key (BookingID), partial dependency not applicable. RideID, PassengerID, BookingTime, and Status all fully depend on BookingID.

3NF: No transitive dependencies. Status depends on the booking itself (BookingID), not on RideID or PassengerID.

Changes Made: None.

---

6. REVIEWS

1NF: All columns are atomic. ReviewID is the unique primary key. Rating, Comment, and ReviewDate are single-valued per row.

2NF: Single-column primary key (ReviewID), partial dependency not applicable. All attributes fully depend on ReviewID.

3NF: No non-key column depends on another non-key column. DriverID and PassengerID both reference USERS but depend on ReviewID, not on each other.

Changes Made: None.

---

7. PAYMENTS

1NF: All columns are atomic. PaymentID is the unique primary key. No repeating groups exist.

2NF: Single-column primary key (PaymentID), partial dependency not applicable. All attributes fully depend on PaymentID.

3NF: No transitive dependencies. Amount, Method, Status, and PaymentDate all depend directly on PaymentID, not on BookingID or each other.

Changes Made: None.

---

8. IDENTITY_VERIFICATION

1NF: All columns are atomic. VerificationID is the unique primary key. DocumentType and DocumentNumber are single-valued per row.

2NF: Single-column primary key (VerificationID), partial dependency not applicable.

3NF: No transitive dependencies. DocumentNumber does not determine any other column — all attributes depend directly on VerificationID.

Changes Made: None.

---

9. VEHICLE_REGISTRATION

1NF: All columns are atomic. RegistrationID is the unique primary key. No multi-valued attributes exist.

2NF: Single-column primary key (RegistrationID), partial dependency not applicable.

3NF: No transitive dependencies. ExpiryDate and Status depend on the registration record itself (RegistrationID), not on VehicleID or RegistrationNumber.

Changes Made: None.

---

10. VERIFICATION_CHECKLIST

1NF: All columns are atomic. CheckListID is the unique primary key. Each checklist item (IdentityVerified, VehicleRegistered, ProfileCompleted, PaymentSetup) is stored as a separate column with a single value.

2NF: Single-column primary key (CheckListID), partial dependency not applicable.

3NF: No transitive dependencies. All checklist columns depend directly on CheckListID and not on each other or on UserID.

Changes Made: None.

End of Normalization Document — Ride Sharing System

3. Dataflow Description

This section describes how data enters, moves through, and exits the Ride Sharing System database.

Data Entry Points:

- A new user registers → data enters the USERS table
- User submits identity documents → data enters IDENTITY_VERIFICATION
- Driver adds a vehicle → data enters VEHICLES
- Vehicle registration details are submitted → data enters VEHICLE_REGISTRATION
- User checklist is updated → data enters VERIFICATION_CHECKLIST
- Driver creates a ride → data enters RIDES, referencing USERS, VEHICLES, and LOCATIONS
- Passenger books a ride → data enters BOOKINGS, referencing RIDES and USERS
- Payment is made for a booking → data enters PAYMENTS, referencing BOOKINGS
- Passenger submits a review → data enters REVIEWS, referencing RIDES and USERS

Data Flow Through Tables:

- USERS is the central table — almost every other table references it
- LOCATIONS feeds into RIDES as origin and destination
- VEHICLES feeds into RIDES as the vehicle used
- RIDES feeds into BOOKINGS and REVIEWS
- BOOKINGS feeds into PAYMENTS

Data Output:

- Ride search results for passengers
- Booking history per user
- Payment records per booking
- Driver ratings from reviews
- Verification status of users and vehicles